

GUIDA ALLO SVOLGIMENTO DEGLI ESERCIZI SUL PASSAGGIO DI PARAMETRI

Gli esercizi sul passaggio di parametri presenti nelle tracce d'esame sono relativamente semplici da svolgere. Tuttavia occorre prestare la massima attenzione durante lo svolgimento poiché basta sbagliare un solo numero per falsare buona parte del risultato.

Iniziamo con una spiegazione teorica per poi passare allo svolgimento di un esercizio.

Nella traccia viene fornito un programma del quale occorre scrivere l'output e disegnare gli stack di attivazione per ogni caso richiesto

Gli elementi da valutare durante lo svolgimento dell'esercizio sono essenzialmente 2, cioè lo **SCOPING** ed il **MODE**

SCOPING

Il tipo di **SCOPING** indica l'ambiente **non locale** (cioè le variabili non dichiarate o ricevute come parametro all'interno della procedure) al quale ciascuna procedure del programma deve fare riferimento. Lo scoping può essere di 2 tipi:

- **SCOPING STATICO:** Per le variabili non locali, la procedure deve fare riferimento al blocco della procedure che la contiene sintatticamente. Per fare questo è sufficiente dare un'occhiata all'indentazione del codice per capire subito a quale blocco occorre fare riferimento.
- **SCOPING DINAMICO:** Per le variabili non locali, la procedure deve fare riferimento al blocco della procedure chiamante

MODE

Il tipo di **MODE** indica la modalità di passaggio dei parametri al momento della chiamata. I tipi di MODE possibili sono 6:

- **IN/OUT PER COPIA:** la variabile passata dal chiamante viene copiata in una variabile locale della procedura chiamata e ciascuna modifica apportata ha effetto esclusivamente sulla variabile locale della procedura chiamata. Quando la procedura chiamata termina, il valore della variabile viene restituito al chiamante e copiato nella relativa variabile locale del chiamante.
- **IN PER COPIA:** la variabile passata dal chiamante viene copiata in una variabile locale della procedura chiamata e ciascuna modifica apportata ha effetto esclusivamente sulla variabile locale della procedura chiamata.
- **IN/OUT PER RIFERIMENTO:** nella variabile della procedure chiamata viene copiato un riferimento alla variabile della procedure chiamante. La variabile del chiamante è accessibile sia in "lettura" che in "scrittura" e qualunque modifica apportata nella procedure chiamata ha immediatamente effetto sul valore della variabile della procedure chiamante.
- **IN PER RIFERIMENTO:** nella variabile della procedure chiamata viene copiato un riferimento alla variabile della procedure chiamante. La variabile del chiamante è accessibile solo in "lettura" e non può essere dunque modificata.
ATTENZIONE: quando il MODE è di tipo *IN PER RIFERIMENTO*, non è possibile modificare il valore della variabile all'interno della procedure chiamata. L'istruzione della procedure chiamata che tenta di modificare una variabile passata dal chiamante con modalità *IN PER RIFERIMENTO*, causa un errore.

Per i MODE **OUT per copia** ed **OUT per riferimento** riporto quanto scritto nelle slide del docente:

Alcuni linguaggi assumono che i parametri OUT non siano inizializzati e ne proibiscono la “lettura”, ad esempio:

- uso a destra di un assegnamento
- passaggio a un parametro *IN* o *IN/OUT* di un'altra procedura

Non esistono regole generali nemmeno tra diverse versioni di uno stesso linguaggio:

- Ada 83 proibisce di “leggere” i parametri OUT
- Le versioni successive invece lo permettono

NOTA: negli esercizi proposti dal docente, la lettura di parametri OUT non inizializzati genera errore.

ESERCIZIO

Passiamo adesso a mostrare un esercizio svolto.

L'esercizio fa parte della prova d'esame del 2 marzo 2017:

Esercizio sul passaggio di parametri – Max 11 punti

Dire qual è l'output del seguente programma nei casi elencati qui sotto:

1. Scoping dinamico, [MODE] = IN per copia
2. Scoping dinamico, [MODE] = IN per riferimento
3. Scoping statico, [MODE] = IN per copia
4. Scoping statico, [MODE] = IN OUT per riferimento

Mostrare gli stack di attivazione (pena la perdita di punti), tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore.

```
program p1
int x; int y; int z; int t;
  procedure p2([IN x copia] int y,[IN x copia] int x)
    int z;
    BEGIN
      z=x;
      x=4;
      t=z-1;
      write(x,y,z,t);
    END

    procedure p3([MODE] int x,[IN x copia] int t)
      int z;
      procedure p4([IN x copia] int x)
        int t;
        BEGIN
          t=x-2;
          z=4;
          y=y;
          p2(x, t);
          write(x,y,z,t);
        END

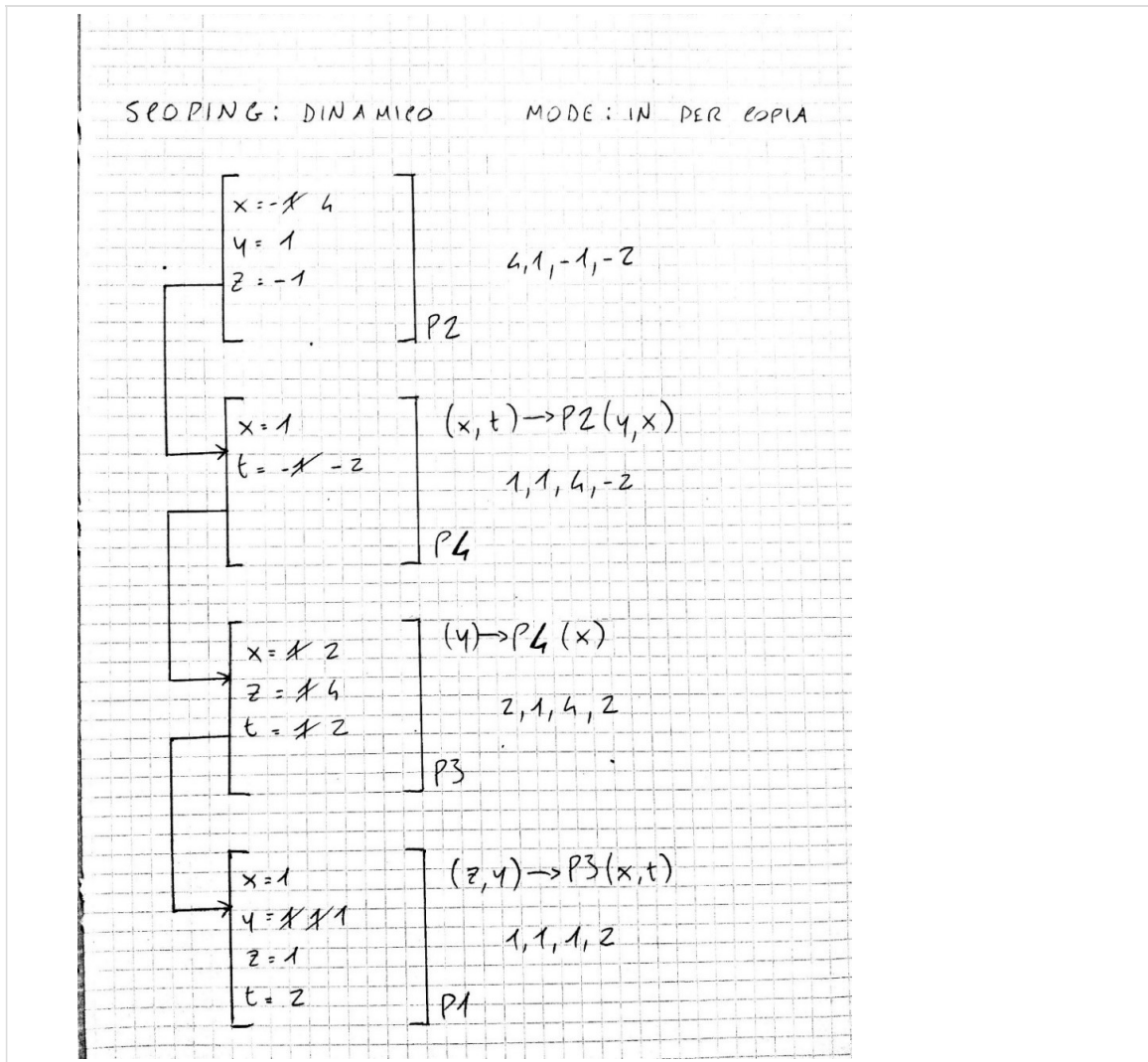
        BEGIN
          z=t;
          x=z*2;
          t=x;
          y=z*1;
          p4(y);
          write(x,y,z,t);
        END

      BEGIN
        x=1;
        y=1;
        z=1;
        t=2;
        p3(z, y);
        write(x,y,z,t);
      END
```

L'esercizio richiede di indicare l'output del programma e disegnare gli stack di attivazione per 4 diverse combinazioni di **SCOPING** e **MODE**.

NOTA: negli svolgimenti seguenti sono già scritti tutti i valori assunti dalle variabili. Chiaramente, partendo dall'inizio dell'esercizio, il valore delle variabili viene aggiornato di volta in volta durante lo svolgimento.

PUNTO 1- SCOPING: dinamico **MODE:** IN per copia



Innanzitutto spieghiamo gli elementi utilizzati in questa rappresentazione grafica degli stack di attivazione.

Ciascuna procedura è rappresentata con un blocco formato da due parentesi quadre.

Le frecce alla sinistra di ogni blocco indicano lo SCOPING. Trattandosi in questo caso di scoping dinamico è ovvio che ogni freccia andrà verso il blocco immediatamente più in basso (il blocco del chiamante)

Le lettere nel blocco rappresentano le variabili ed i relativi valori da esse assunti durante l'esecuzione del programma.

Alla destra del blocco, dal basso verso l'alto, si trovano il nome della procedura, l'output della procedura e la corrispondenza delle variabili di procedura chiamante e chiamata (quest'ultima io la scrivo per avere più informazioni sotto mano ma volendo si può vedere direttamente dalla traccia mentre si svolge l'esercizio).

Andiamo ad analizzare i blocchi delle singole procedure:

- **PROCEDURE P1:** La procedura P1 è la prima ad essere eseguita. Le variabili x, y, z, t vengono inizializzate rispettivamente ai valori $1, 1, 1, 2$ e viene chiamata la procedura P3 passando i valori z, y di P1 alle variabili x, t di P3

- **PROCEDURE P3:** Il chiamante P1 passa a P3 i valori (1,1) che vengono copiati nelle variabili (x,t) con modalità IN per copia ed anche la variabile z viene dichiarata localmente all'interno del blocco. Per la variabile non locale y, invece, occorre fare riferimento allo scoping. Trattandosi in questo caso di scoping dinamico, va considerata la variabile y del chiamante, e quindi la y del blocco P1. All'inizio del blocco saranno quindi inizializzate soltanto le variabili x=1 e t=1.

Vediamo ora come cambia il valore delle variabili per ogni istruzione:

z=t ---> alla variabile locale z viene assegnato il valore della variabile t, quindi sarà z=1

x=z*2 ---> analogamente il valore di x sarà $x=1*2$ cioè x=2

t=x ---> t=2

y=z*1 ---> y=1 come detto, y non è una variabile locale poichè non è stata dichiarata nel blocco P3, dunque si fa riferimento allo scoping e si modifica il valore della y del blocco P1 (che era già 1 ma va barrato e riscritto).

A questo punto le variabili x,z,t di P3 hanno valore 2,1,2 e viene chiamata la procedure P4 passando il valore di y alla variabile x di P4

- **PROCEDURE P4:** Il chiamante P3 passa il valore 1 della variabile y alla variabile locale x di P4 con modalità IN per copia. Come detto la variabile y non è una variabile locale di P3 e quindi a P4 viene passato il valore della y di P1 a cui P3 fa riferimento per lo scoping. Localmente viene dichiarata anche la variabile t, mentre per le variabili y e z si dovrà fare riferimento alle variabili omonime di P3 in virtù dello scoping dinamico.

Analizziamo le istruzioni:

t=x-2 ---> t=-1

z=4 ---> non essendo una variabile locale, va modificata la z di P3 e posta uguale a 4

y=y ---> la y di P4 non è una variabile locale e si fa riferimento alla y di P3 che a sua volta fa riferimento alla y di P1. Dunque questa assegnazione assegna ancora una volta il valore 1 alla variabile y di P1.

A questo punto viene effettuata una chiamata a P2 passando i valori di (x,t) alle variabili (y,x) di P2

- **PROCEDURE P2:** La procedure P2 ha come variabili locali x,y,z. I valori di y e x sono passati dal chiamante e sono y=1 e x=-1. Analizziamo le istruzioni:
z=x ---> z=-1
x=4
t=z-1 ---> t non è una variabile locale e dunque va modificata la variabile t del chiamante P4, quindi $t=-1-1$ cioè t=-2. Da notare che la variabile z utilizzata in questa istruzione è comunque la variabile locale z di P2. Sottolineiamo che lo scoping riguarda solo ed esclusivamente le variabili non locali.

A questo punto, andando a ritroso tutte le procedure eseguono la **write(x,y,z,t)** stampando l'output che, guardando l'ultimo valore assunto da ciascuna variabile, sarà il seguente:

P2= 4,1,-1,2

P4=1,1,4,-2

P3=2,1,4,2

P1=1,1,1,2

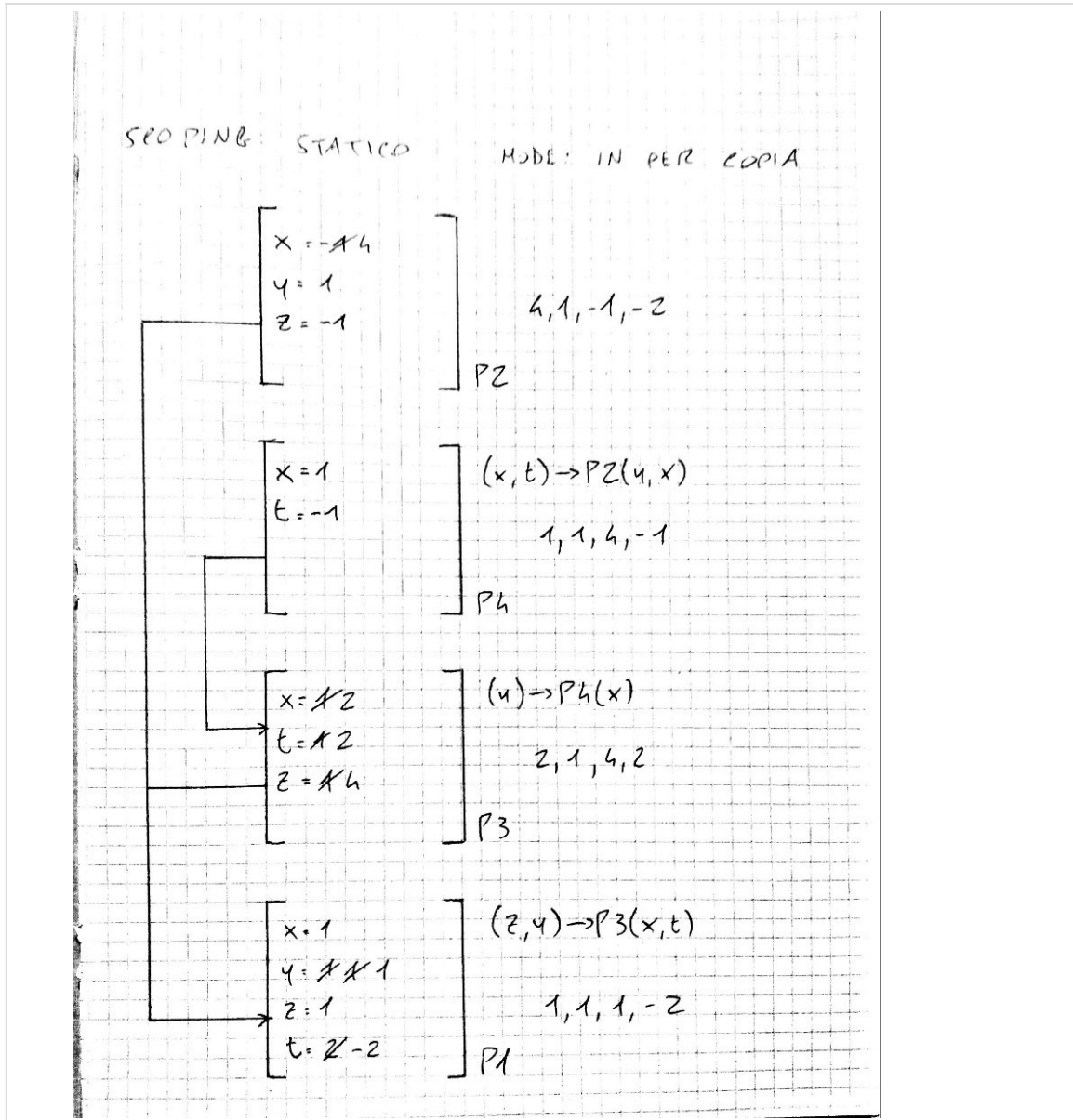
NOTA: anche per la write(x,y,z,t) si fa riferimento allo scoping per le variabili non locali.

PUNTO 2- SCOPING: dinamico **MODE:** IN per riferimento

Il **99,9%** delle volte, negli esercizi sul passaggio di parametri, il programma della traccia è scritto in modo da dare errore nel caso di passaggio con modalità IN per riferimento. Come detto all'inizio, un'istruzione che tenta di modificare una variabile passata con modalità **IN PER RIFERIMENTO**, causa un errore. In questi casi quindi, basta controllare se c'è una istruzione che tenta di modificare una variabile ricevuta con modalità IN per riferimento. Nell'esercizio dell'esempio, occorre guardare la procedure P3 e più precisamente la sua variabile **x**. Se troviamo un'istruzione che tenta di modificare il valore di tale variabile, allora possiamo dire che

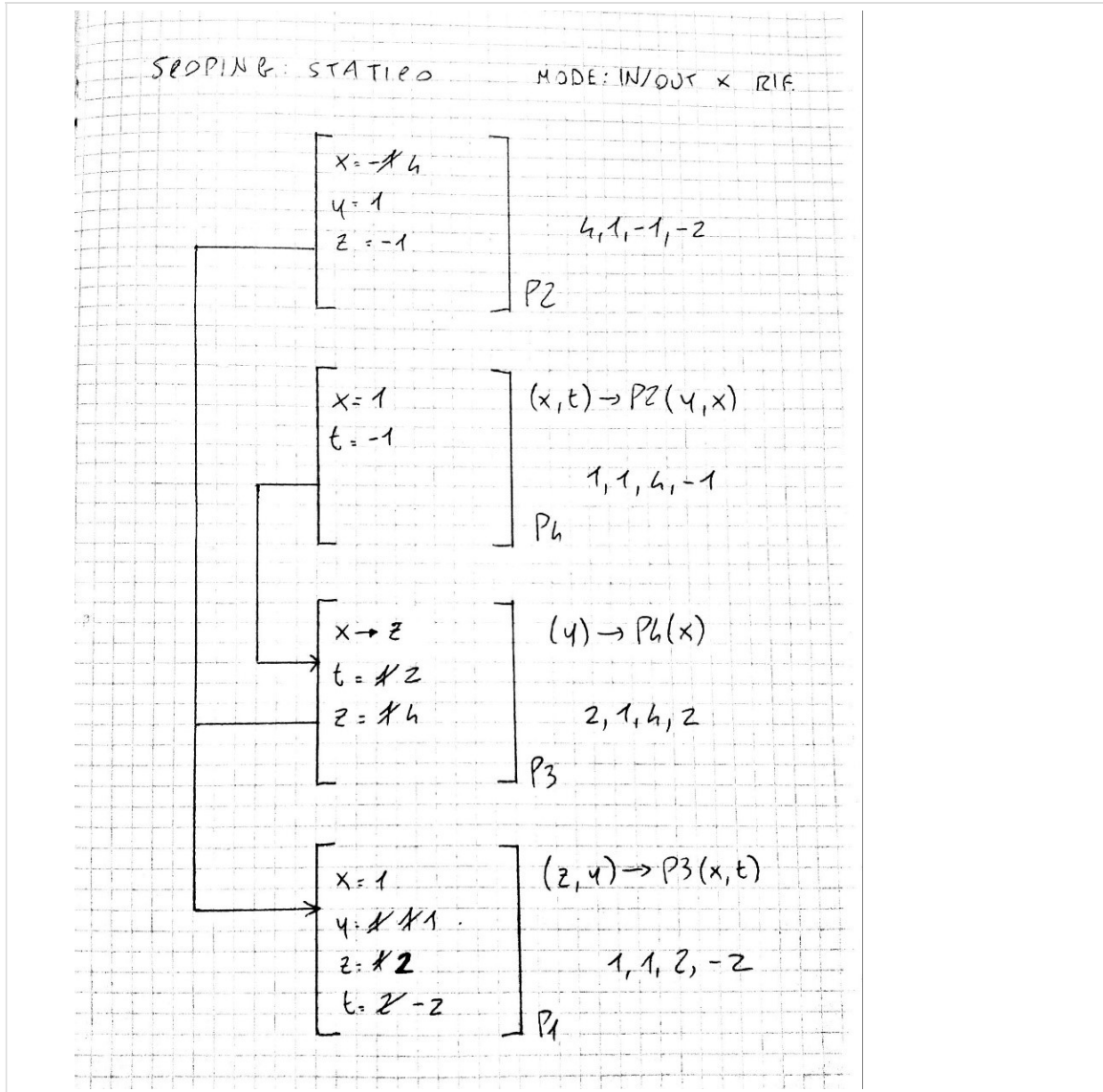
quella istruzione causa errore. Nel nostro caso quindi l'istruzione $x=z*2$ di P3 causa errore e **NON** vanno indicati output e stack di attivazione.

PUNTO 3- SCOPING: statico **MODE:** IN per copia



Lo svolgimento di questo punto è del tutto simile a quello del punto 1. L'unica differenza sta nel tipo di SCOPING. In questo caso lo scoping è di tipo statico e dunque per le variabili non locali ogni procedura deve fare riferimento al blocco che la contiene sintatticamente. Dando uno sguardo all'indentazione del codice si capisce subito che P2 e P3 sono contenute in P1 mentre P4 è contenuta in P3. Quindi, durante lo svolgimento, aggiungiamo le frecce alla sinistra dei blocchi per poter vedere subito lo scoping di ciascun blocco senza dover ogni volta ricontrollare la traccia. In questo caso, per esempio, l'assegnamento $t=z-1$ nel blocco P2 va a modificare la variabile t del blocco P1.

PUNTO 4- SCOPING: statico **MODE:** IN/OUT per riferimento



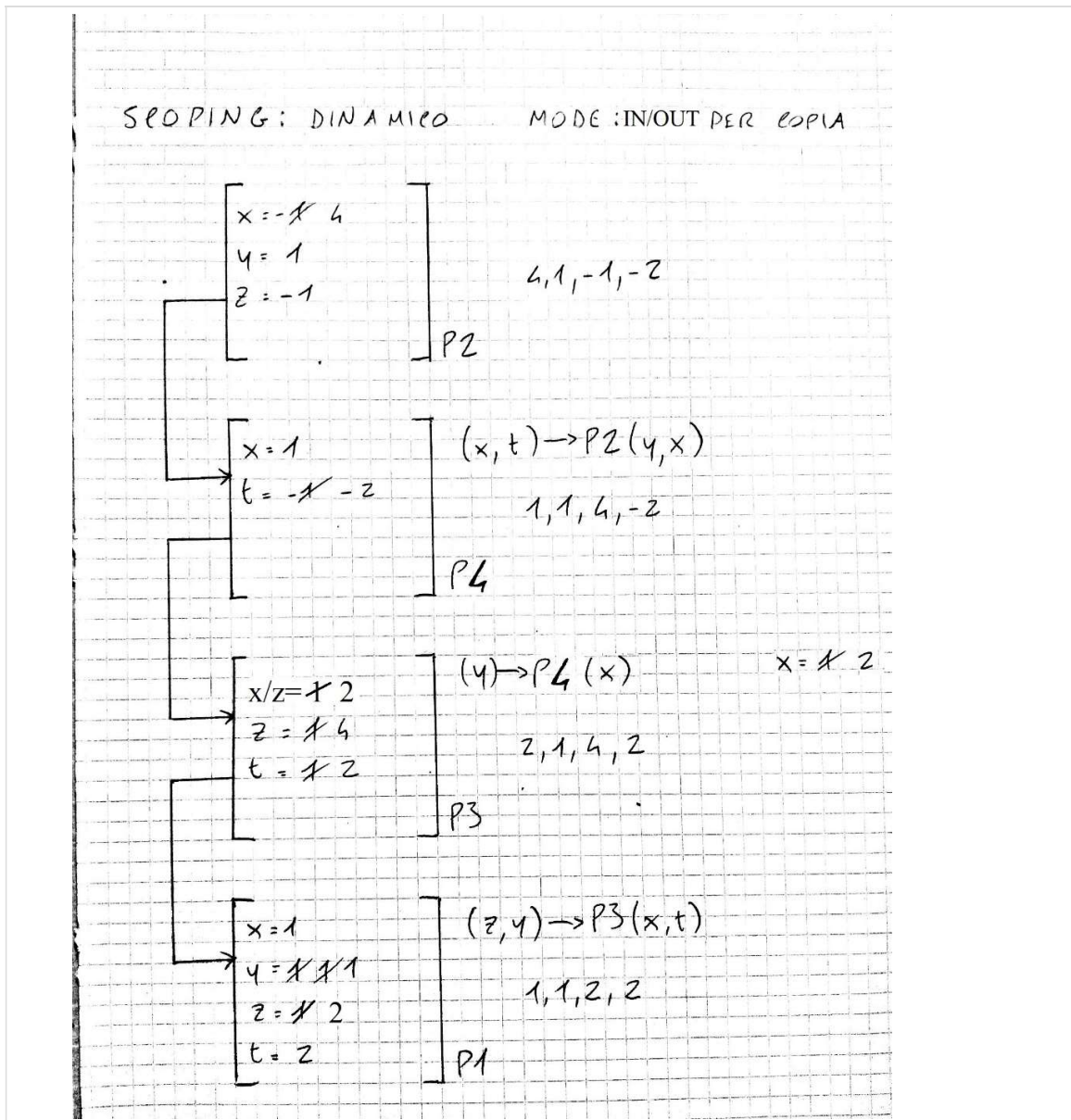
Nel blocco P3 ho utilizzato la scrittura $x \rightarrow z$ per indicare che la variabile locale x contiene un riferimento alla variabile z del chiamante (cioè P1).

Anche in questo caso lo svolgimento è analogo a quello dei punti precedenti. L'unica differenza sta nella modalità di passaggio della variabile x di P3 che è di tipo IN/OUT per riferimento. In questo caso qualsiasi modifica della variabile x di P3 comporta l'immediata modifica della variabile z di P1 a cui fa riferimento. Per esempio, l'assegnamento $x = z * 2$ in P3 comporta l'assegnamento del valore $z * 2$ alla variabile z di P1.

ATTENZIONE: occorre stare attenti a non confondere lo scoping delle variabili non locali con il riferimento delle variabili passate con modalità IN/OUT per riferimento.

Poiché nell'esercizio precedente non ci sono le modalità di passaggio **IN/OUT per copia**, **OUT per copia** ed **OUT per riferimento**, ne riporto un esempio a titolo esplicativo.

Supponiamo che il punto 1 dell'esercizio fosse stato:
 SCOPING: dinamico e MODE: IN/OUT per copia
 Lo svolgimento dell'esercizio sarebbe stato il seguente:



La scrittura **x/z** nel blocco P3 indica che al termine della procedura P3 il valore della variabile x deve essere copiato nella variabile z del chiamante, in questo caso P1. Quindi durante lo svolgimento dell'esercizio, dopo aver scritto l'output del blocco P3 assegneremo alla variabile z di P1 il valore della variabile x di P3, quindi z=2

Vediamo due casi di passaggio OUT per riferimento:

- **CASO CON ERRORE (PUNTO 4):**

Dire qual è l'output del seguente programma nei casi elencati qui sotto:

1. Scoping dinamico, [MODE] = IN per copia
2. Scoping statico, [MODE] = IN per copia
3. Scoping statico, [MODE] = IN OUT per riferimento
4. Scoping statico, [MODE] = OUT per riferimento

Mostrare gli stack di attivazione (pena la perdita di punti), tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore.

```
program p1
int a; int b; int c; int d;
procedure p2([IN OUT x copia] int a)
int c; int d;
BEGIN
if b>3 then c=b else c=a+2;
d=4;
a=c-4;
if a>10 then b=c else b=d-3;
write(a,b,c,d);
END

procedure p3([MODE] int b,[IN x copia] int c)
int d;
procedure p4([IN x copia] int d)
int a; int c;
BEGIN
a=b*4;
c=3;
d=b-4;
b=d;
p2(b);
write(a,b,c,d);
END

BEGIN
d=a;
b=b+2;
c=a*3;
a=1;
p4(b);
write(a,b,c,d);
END

BEGIN
a=0;
b=3;
c=0;
d=2;
p3(d, c);
write(a,b,c,d);
END
```

In questo caso, l'assegnamento **b=b+2** nel blocco P3 è **illegale** perchè si tenta di leggere il valore della variabile **b** che non è inizializzata.

- **CASO SENZA ERRORE (PUNTO 2):**

Dire qual è l'output del seguente programma nei casi elencati qui sotto:

1. Scoping dinamico, [MODE] = IN per copia
2. Scoping dinamico, [MODE] = OUT per riferimento
3. Scoping statico, [MODE] = IN per copia
4. Scoping statico, [MODE] = IN OUT per copia

Mostrare gli stack di attivazione (pena la perdita di punti), tranne nei casi di errore, nei quali bisogna invece indicare l'istruzione che causa l'errore.

```
program p1
int q; int r; int s; int t;
  procedure p2([IN x copia] int s,[IN x copia] int q)
    int r;
      procedure p3([IN OUT x rif] int r)
        int q;
        BEGIN
          q=r*3;
          r=r+2;
          s=2;
          t=q;
          p4(t, s);
          write(q,r,s,t);
        END

      BEGIN
        if s<5 then r=q*2 else r=4;
        s=r;
        q=1;
        t=3;
        p3(t);
        write(q,r,s,t);
      END

    procedure p4([MODE] int q,[IN x copia] int t)
      int s;
      BEGIN
        s=4;
        q=2;
        t=s+4;
        r=t;
        write(q,r,s,t);
      END

    BEGIN
      q=4;
      r=3;
      s=0;
      t=1;
      p2(r, q);
      write(q,r,s,t);
    END
```

In questo caso non c'è errore poichè non c'è nessuna lettura del valore della variabile q non inizializzata. Lo svolgimento, in questo caso, è del tutto uguale alla modalità IN/OUT per riferimento.

Ho controllato tutte le prove presenti sul sito del docente ma non ho trovato alcuna traccia contenente la modalità OUT per copia, quindi mi limito a ribadire quanto detto in precedenza, cioè che l'eventuale lettura di variabili non inizializzate genera errore.

Per completare la guida riporto questa tabella fatta da **luigi_starace** che specifica quali errori possono verificarsi per ogni modalità di passaggio dei parametri:

MODALITÀ	Errore in compilazione
IN PER COPIA	MAI
IN PER RIFER.	Se c'è una scrittura sul parametro
OUT PER COPIA	Se c'è una lettura del valore del parametro
OUT PER RIFER.	Se c'è una lettura del valore del parametro
IN/OUT PER COPIA	MAI
IN/OUT PER RIFER.	MAI

STRUCTURAL E NAME EQUIVALENCE

Ci sono 2 modi standard per determinare se due tipi sono uguali: name equivalence and structural equivalence.

NAME EQUIVALENCE:

La più lineare è la name equivalence: 2 tipi sono uguali se e solo se hanno lo stesso nome.

CODICE:

```
typedef struct {
    int vettore[5];
    int i;
} tipoUNO;

typedef struct {
    int vettore[5];
    int i;
} tipoDUE;

tipoUNO x, y;
tipoDUE r, s;
```

Con la name equivalence, x ed y saranno lo stesso tipo e anche r ed s saranno dello stesso tipo, MA
x,y non sono dello stesso tipo di ad r,s

Assegnamenti validi:

CODICE:

```
x = y;  
r = s;
```

Assegnamenti non valido

CODICE:

```
x = r;  
y = s;
```

STRUCTURAL EQUIVALENCE:

Due tipi sono uguali se e solo se hanno la stessa struttura che può essere interpretata in modi differenti:

Una interpretazione restrittiva sarebbe che i nomi ei tipi di ciascun componente dei due tipi devono essere uguali e devono essere elencati nello stesso ordine nella definizione del tipo.

Un requisito meno rigoroso sarebbe che i tipi di componenti devono essere uguali e nello stesso ordine in due tipi, ma i nomi dei componenti potrebbe essere diverso.

Come sopra, i due tipi dichiarati tipoUNO tipoDUE saranno considerati equivalenti.

Questo vuol dire che l'interprete o il compilatore accetterà uno statement tipo:

CODICE:

```
x = r;
```

C non supporta la structural equivalence e avrebbe dato errore per questo tipo di assegnamento.

Tranne in questo caso:

Esercizio 3: [2 punti] (Barrare tutte le risposte corrette) Scrivere delle dichiarazioni in C o C++ tali che l'assegnamento `y = x`: rispetti la structural equivalence ma *non* la name equivalence e sia corretto nel linguaggio scelto.

```
ad es.  
typedef int T1;  
typedef int T2;  
T1 x;  
T2 y;
```

In parole povere, la naming equivalence nasce dal fatto che si presuppone che se un programmatore ha definito 2 tipi di dato differenti, voleva appunto intenderli come differenti. Altrimenti non avrebbe senso usare i typedef.

INTERFACCE E REALIZZAZIONI

Un'interfaccia viene modellata allo stesso modo in cui viene modellato il comportamento di una classe e rappresenta un insieme di operazioni che una classe offre ad classi. lo scopo principale delle interfacce è appunto offrire queste operazioni o funzioni ad altre classi che le possono implementare.

Un'interfaccia non ha attributi ma soltanto operazioni (metodi). In UML per rappresentare le interfacce si utilizza un rettangolo con la dicitura «interface» o un piccolo cerchio (notazione lollipop, “a leccalecca”).

La relazione tra una classe ed un'interfaccia viene definita realizzazione. Tale relazione è visualizzata nel modello da una linea tratteggiata con un triangolo largo aperto costruito sul lato dell'interfaccia o con una linea nella notazione compatta lollipop.

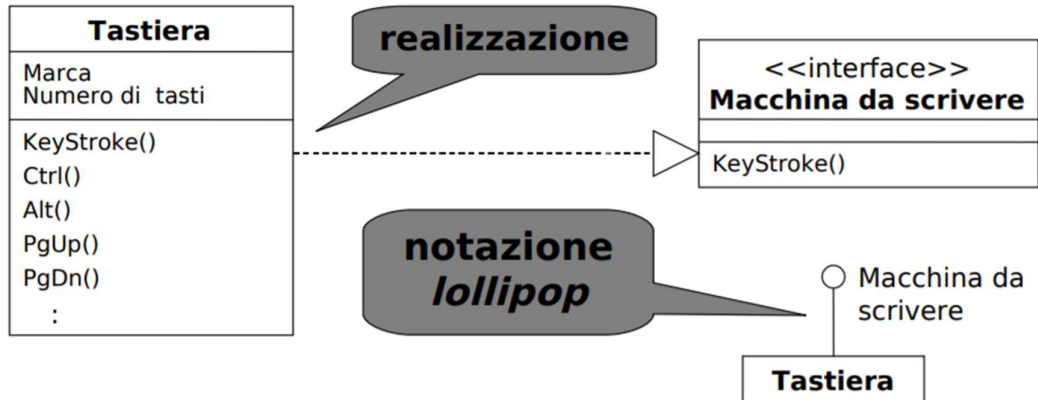
Solitamente si utilizza un'interfaccia quando le funzionalità che mette a disposizione non potrebbero essere inserite in una classe padre. Per spiegare questo concetto facciamo un breve esempio: immaginiamo un metodo *mangia()* che è comune sia ad un essere umano che ad un serpente. Per esempio non sarebbe possibile inserire il metodo *mangia()* in una classe *animale* e poi fare una derivazione in quanto l'essere umano non è un animale. Per questo motivo è possibile definire un'interfaccia che ha il metodo *mangia()* e poi effettuare due realizzazione una per l'essere umano e una per il serpente.

Il vantaggio dell'uso dell'interfaccia è dunque quello di non introdurre dipendenze tra classi che non ne hanno, ma comunque effettuare una sorta di collegamento logico.

Altro vantaggio che le interfacce offrono è quello di suddividere il problema in sotto-problemi dove ogni sotto-problema non è minimamente a conoscenza di come è strutturato un altro e la comunicazione avviene appunto tramite le interfacce. Un sistema progettato in questo modo ha il vantaggio di essere facilmente estendibile in quanto è possibile aggiungere nuovi moduli e collegarli agli altri tramite le interfacce evitando di collegare, in maniera diretta, i vari moduli.

ESEMPIO:

La tastiera del computer è un tipico esempio di realizzazione di una interfaccia. La pressione di un tasto (KeyStroke) rappresenta un'operazione che è stata definita dall'interfaccia Macchina per scrivere. L'operazione KeyStroke() viene realizzata anche sulla tastiera dei computer. D'altra parte sulla tastiera dei computer si trovano un insieme di operazioni che non appartengono alla macchina per scrivere (Ctrl, Alt, PageUp, PageDown, ecc.) .



SEQUENCE DIAGRAM

Un **Sequence Diagram** (in italiano: **Diagramma di sequenza**) è un diagramma previsto dall'[UML](#) utilizzato per descrivere uno scenario.

Uno scenario è una determinata sequenza di azioni in cui tutte le scelte sono state già effettuate; in pratica nel diagramma non compaiono scelte, né flussi alternativi.

Il *Sequence Diagram* descrive le relazioni che intercorrono, in termini di messaggi, tra Attori, Oggetti di business, Oggetti o Entità del sistema che si sta rappresentando.

Messaggi:

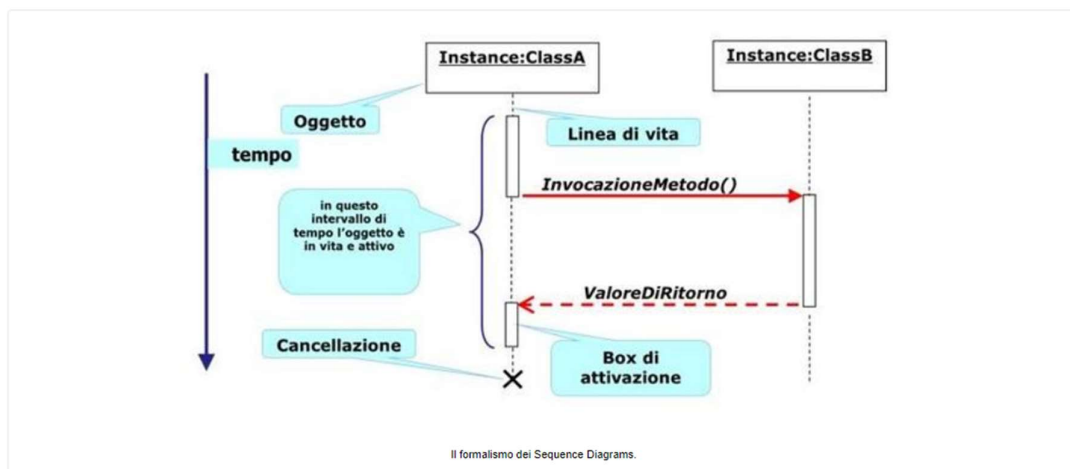
Un messaggio è un'informazione che viene scambiata tra due entità (oggetti). Solitamente chi invia il messaggio, la parte attiva, è l'attore.

Il messaggio è sincrono, se l'emittente rimane in attesa di una risposta, o asincrono, nel caso l'emittente non aspetti la risposta e questa può arrivare in un secondo momento.

Il messaggio che viene generato in risposta ad un precedente messaggio, al quale si riferisce anche come contenuto informativo, è detto messaggio di risposta.

Un messaggio, in cui il ricevente è nello stesso tempo l'emittente, è detto ricorsivo.

Sequence Diagrams



Messaggi nei Sequence Diagrams

- Se l'oggetto che invia il messaggio rimane in attesa che l'oggetto ricevente ritorni, si ha un messaggio sincrono 
- Se l'oggetto che invia il messaggio prosegue la propria elaborazione in parallelo all'oggetto chiamato, siamo in presenza di un messaggio asincrono. 
- Il valore restituito all'oggetto chiamante si indica con un messaggio di ritorno 

Diagramma:

Sequence Diagram



POLIMORFISMO PARAMETRICO E PER INCLUSIONE

Il polimorfismo è la capacità di cose diverse di apparire nella stessa forma in un determinato contesto. Si supponga per esempio di scrivere una [funzione](#) che restituisce il massimo fra due interi:

```
int max(int a, int b)
{
    if(a > b) return a;
    else return b;
}
```

Se si prova a invocare la funzione max() per determinare il massimo tra due double, il [compilatore](#) segnalerà un errore. Nel caso dell' esempio, per determinare il massimo tra due double sarebbe necessario scrivere un' altra funzione:

```
double maxd(double a, double b)
{
    if(a > b) return a;
    else return b;
}
```

Si noti che il corpo delle due funzioni è formalmente identico: quello che cambia è solo la dichiarazione. Un [linguaggio di programmazione](#) che supporta il polimorfismo fornisce al programmatore dei mezzi per evitare di dover scrivere metodi distinti in questo tipo di casi. Si possono distinguere quattro tipi di polimorfismo, tratterem solo quello parametrico e per inclusione.

- *parametrico*: le funzioni o gli operatori sono parametrizzati secondo il tipo a cui possono essere applicati Negli esempi riportati sopra, al posto del tipo int, double,...si può utilizzare una variabile T, che può assumere valori int, double, ecc....
- *per inclusione*: un *metodo* che può essere applicato a tutti gli [oggetti](#) inclusi nella *classe* che lo ammette. (questo tipo di polimorfismo è quello che sta alla base della [programmazione orientata agli oggetti](#)).

Nell' approccio orientato agli oggetti, che utilizza il polimorfismo per inclusione, nel caso dell' esempio: anziché definire tante funzioni diverse che restituiscano il massimo tra due oggetti di ciascun tipo, la programmazione ad oggetti preferisce definire la classe degli elementi su cui ha senso applicare il metodo max(). Una volta definita questa classe tutte le classi da essa derivate ne ereditano i metodi.

Pro del polimorfismo parametrico:

- Non richiede controlli di tipo a run time
- Anticipa scoperta errori di tipo a tempo di compilazione

Pro del polimorfismo per inclusione:

- Permette strutture dati eterogenee
 - Ad esempio, Stack di elementi di tipo diverso
- Se in Java si usa unicamente il polimorfismo parametrico, allora tutti i controlli di tipo possono avvenire a tempo di compilazione.

HEAP E STACK

- lo **Stack** è un' area di memoria utilizzata per l'esecuzione dei metodi e viene utilizzato appunto per passare i parametri ai metodi e per memorizzare provvisoriamente i risultati restituiti dalla loro invocazione.

- L' **heap** è un'area di memoria utilizzata per l'allocazione degli oggetti istanziati e su di esso opera il **garbage collector** (raccoglitore di spazzatura) che è in grado di rilevare gli oggetti inutilizzati e distruggerli

Nei linguaggi a oggetti l'ambiente non locale di un metodo si può trovare nello heap e nella zona statica, mentre quello locale nello stack.

PARADIGMA LOGICO E FUNZIONALE

Paradigma logico:

Il paradigma di programmazione logico utilizza un approccio dichiarativo per la soluzione dei problemi. Vengono fatte delle asserzioni logiche, cioè dichiarazioni che forniscono una definizione del problema stabilendo i fatti conosciuti. Le dichiarazioni sono costruite attraverso le relazioni. Per esempio ci possono essere dichiarazioni come le seguenti: Mario è il padre di Luca (1) padre (Luca) = Mario Luca è il padre di Angelo (2) padre (Angelo) = Luca Ci può essere anche la relazione: (3) nonno(x) = padre (padre (x)) Poi vengono preparate le interrogazioni, a partire dalle quali il sistema di risoluzione, mediante passaggi logici, restituisce i risultati in output. Per esempio, usando le dichiarazioni precedenti possiamo chiedere: chi è il nonno di Angelo? Attraverso passaggi logici e cercando tra le dichiarazioni, il programma restituisce il nome Mario. I passaggi che portano alla soluzione cominciano dall'interrogazione che può essere espressa in questa forma: nonno(Angelo) = ? Il sistema di risoluzione cerca tra le dichiarazioni e le relazioni una possibile associazione. Trova un'associazione tra il membro sinistro dell'interrogazione e quello della relazione (3). Con questa relazione si può effettuare la sostituzione $x = \text{Angelo}$ e si ottiene: nonno(Angelo) = padre (padre (Angelo)) = ? A questo punto si può utilizzare la dichiarazione (2) ed effettuare la seguente sostituzione: nonno(Angelo) = padre (padre (Angelo)) = padre (Luca) = ? L'ultimo passaggio viene effettuato con la dichiarazione (1) che permette di inferire nonno(Angelo) = padre (padre (Angelo)) = padre (Luca) = Mario e quindi nonno(Angelo) = Mario. Il vantaggio di questa modalità di programmazione consiste nella possibilità di elencare le relazioni e definire le interrogazioni, lasciando poi al sistema il compito di risolvere il problema, senza dover programmare altre procedure.

Paradigma funzionale:

Niente variabili! Niente assegnamenti!

Il paradigma di programmazione funzionale ha come base l'uso delle funzioni, intese in senso matematico, per risolvere i problemi. Le funzioni ricevono in input certi valori e restituiscono un risultato. Si può sfruttare la composizione funzionale, usando cioè una funzione (il suo risultato) come input di un'altra funzione. Il sistema che implementa il paradigma funzionale ha il compito di risolvere la funzione associata al problema e restituire il risultato. Per risolvere questa funzione dovrà valutare le varie funzioni di cui è composta. © Istituto Italiano Edizioni Atlas 2 Come esempio di programmazione funzionale, si pensi al calcolo della funzione $(x+y)^2$, avendo come input la sequenza .

Per risolvere questo problema si definiscono certe funzioni che poi verranno composte tra loro per ottenere il risultato voluto. SOMMA: $= a + b$ Abbiamo definito una funzione che calcola la somma di due valori di input. QUADRATO: $= c * c$ Abbiamo definito una funzione che calcola il quadrato di un numero dato come input. Sfruttando le due funzioni definite precedentemente, possiamo costruire il programma che calcola $(x + y)^2$. PROGRAMMA: =

QUADRATO(SOMMA(x, y)) Avendo come input la sequenza , il risultato si trova eseguendo PROGRAMMA (3, 4). Seguendo passo a passo l'esecuzione ed effettuando le corrette sostituzioni si ottiene: PROGRAMMA(3, 4) = QUADRATO(SOMMA(3, 4)) = QUADRATO(7) = 49. I linguaggi basati sul paradigma funzionale non sono dipendenti dalle istruzioni di assegnamento e non si basano su una sequenza di passi con cui risolvere i problemi. Questa indipendenza dall'ordine di esecuzione fa sì che i linguaggi funzionali siano particolarmente adatti per programmare i calcolatori paralleli. Tra gli svantaggi di questi linguaggi c'è la bassa efficienza.

Paradigmi in pillole

- **Imperativo:** Esecuzione sequenziale che realizza, tramite lo statement di assegnazione, modifiche della memoria: Si pensa il programma come sequenza di azioni. Servono costrutti per iterazioni
- **Funzionale:** I programmi sono funzioni. I "dati" sono gli argomenti su cui si valutano le funzioni. Niente variabili! Niente assegnamenti! dunque niente iterazioni (sarebbero inutili). Rimpiazzate con?...
- **Logico:** I programmi sono teorie logiche. L'esecuzione è la prova nella teoria-programma del teorema il cui enunciato è il problema. Si dice cosa si vuole ottenere, non come ottenerlo. Gli algoritmi possono essere non deterministici (la valutazione no...)
- **Orientato ad oggetti:** Il programma è un insieme di classi ognuna della quali definisce il tipo (cioè operazioni e dominio dei valori possibili) per lo "stato" degli oggetti sue istanze. L' esecuzione è uno scambio di messaggi fra oggetti che induce l'esecuzione di "operazioni" da parte degli oggetti riceventi e possibilmente un cambiamento del loro stato. Nei linguaggi più comuni le operazioni somigliano al modello imperativo. Ma esistono linguaggi O.O. di sapore funzionale e logico
- **Parallelo:** Programmi che descrivono entità distribuite che sono eseguite contemporaneamente

Paradigmi: qualche esempio informale

Approccio imperativo

function definition

```
function factI (n)
  local accumulator = 1
  for i = 1,n do
    accum = accumulator*i
  end
  return accum
end
```

trace of execution

```
factI(4):
  accumulator = 1
  i = 1  accumulator = 1 * 1
  i = 2  accumulator = 1 * 2
  i = 3  accumulator = 2 * 3
  i = 4  accumulator = 6 * 4
  return 24
```

Approccio funzionale

Notare come *n*
assuma il ruolo di
i e...

ricorsione invece
di iterazione

function definition

```
function factR (n)
  if n == 1 then
    (return) 1
  else
    (return) n*factR(n-1)
  end
end
```

trace of execution

```
factR(4) =
4 * factR(3) =
3 * factR(2) =
2 * factR(1) =
  1
  1
  1
  2
  6
  24
```

RACCOLTA SOLUZIONI MEM ED ENV

PER FARLA BREVE: $x=y$ equivale a $--> env(x)=mem(env(y))$

- In C e C++, l'assegnazione le cui parti sinistra e destra denotano $env(x)+1$ e $mem(mem(env(y))+1)$, rispettivamente, è:
 $env(x)+1 = mem(mem(env(y))+1)$
 $x[1] = *(y+1);$
- $env(x)+1 = mem(env(y))+1$
 $x[1] = y+1$
- $mem(env(x)) = mem(env(y)+1)$
 $*x = y[1]$
- Date le dichiarazioni `int x, *y, v[10];`
e sapendo come gli identificatori corrispondono a `mem` e `env`, dire quali degli assegnamenti qui sotto sono corretti in C e C++:
`y = y+1;`
- Nell'assegnamento `*x = *y + 1` dei linguaggi C e C++, l'espressione a DESTRA dell'assegnamento denota:
 $mem(mem(env(y))) + 1$
- Nell'assegnamento `x = *(y + 2)` dei linguaggi C e C++, l'espressione a destra denota:
 $mem(mem(env(y)) + 2)$
- Nell'assegnamento `x = *y + 2` dei linguaggi C e C++, l'espressione a destra denota:
 $mem(mem(env(y))) + 2$
- Considerate l'assegnamento `x=*(y+1)` dove il tipo di `y` è `int*`
 $mem(mem(env(y))+1)$
- Scrivere una espressione C che denoti $mem(env(v)+1)$
`int x, v[ ]; x = v[1];`
`int x, *v; x = *(v+1);`
- Barrare l'interpretazione corretta della espressione `&x` nell'assegnamento:
`int x, *y;`
`y = &x;`

`env(x)`
- in C l'espressione `&x` in `y=&x;` rappresenta $env(x)$
- $env(x)+1 = mem(env(y))+1$
 $x[1] = y+1;$

- `int x, *y;`
`y = &x;`

`mem(env(y)) = env(x)` //non ha soluzione, ma dovrebbe essere così:
`mem(y) = env(x)` //secondo Sebastiano

- Dato il codice C qui riportato:
`int x[10];`
`x[5] = x[6];`

`env(x) + 5 = mem(env(x) + 6)` //(i vettori denotano l'indirizzo iniziale in cui vengono memorizzati)

- Dato il codice C qui riportato, barrare l'espressione che rappresenta la parte sinistra dell'assegnamento:

`int * x;`
`*(x+1) = 5;`

`mem(env(x)) + 1`

- Scrivere una espressione C che denoti
`mem(env(v)+1)`

`v[1];` //valide entrambi. Vedi sol-150219
`*(v+1);` //valide entrambi Vedi sol-150219

- Barrare tutte le espressioni che vengono modificate dall'istruzione `x=y[i++]`;
`mem(env(x))` e `mem(env(i))`

- Nell'istruzione C/C++: `x = *(&(v[2])+3)` il significato della parte destra è
`mem(env(v)+5);` //Da verificare non c'è soluzione

- `mem(env(x)) = mem(env(v))+3`
`*x = *(v+3)`
oppure guardando le possibili soluzioni
`*x = v+3` //notare anche sol-140619

- Nel C e nel C++, l'assegnazione le cui parti destra e sinistra denotano
`env(x) = mem(env(v)+3)`
`x = v+3;` //Fonte Sebastiano

- `*(x+1) = &y` = `mem(env(x))+1 e env(y)+0`
- `mem(env(x))+1 e env(y)+0` = `mem(env(x))+1 e mem(mem(env(y))+0)`
- `x = y[2]` `env(x)` = `mem(env(y)+2)`
- `x = *(z+i)` = `mem( mem(env(z)) + mem(env(i)) )`
- `*(x[1]) = y[0]` = `mem(env(x)+1) e mem(env(y))`